

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА»
(СПбГУТ)**

**АРХАНГЕЛЬСКИЙ КОЛЛЕДЖ ТЕЛЕКОММУНИКАЦИЙ
ИМ. Б.Л. РОЗИНГА (ФИЛИАЛ) СПбГУТ
(АКТ (ф) СПбГУТ)**

СОГЛАСОВАНО

Рук. предприятия

(Подпись) И.А. Бастрыкина
(И.О. Фамилия)

«12» апреля 2026г.

ТЕХНИЧЕСКИЙ ОТЧЁТ по ПМ.02, ПМ.04

АРОО «Федерация тхэквондо ГТФ»

Информационные системы и программирование

09.02.07. 26ТО02. 012 ПЗ

(Обозначение документа)

Студент	<u>ИСПП-21</u>	<u>12.04.2026</u>	<u>В.А. Колосов</u>
	(Группа)	(Подпись)	(Дата)
Рук. практики от предприятия	<u>И.А. Бастрыкина</u>	<u>12.04.2026</u>	<u>И.А. Бастрыкина</u>
	(Подпись)	(Дата)	(И.О. Фамилия)

Архангельск 2026

СОДЕРЖАНИЕ

Перечень сокращений и обозначений	3
Введение.....	4
1 Охрана труда и техника безопасности при работе на ПК.....	6
1.1 Общие требования безопасности	6
1.2 Требования безопасности перед началом работы.....	6
1.3 Требования безопасности во время работы.....	7
1.4 Требования охраны труда в аварийных ситуациях.....	7
1.5 Требования охраны труда по окончании работы	8
2 Выполнение работ по ПМ.11	9
2.1 Проектирование базы данных Ошибка! Закладка не определена.	
2.2 Разработка базы данных и объектов базы данных	Ошибка!
Закладка не определена.	
2.3 Администрирование и защита базы данных Ошибка! Закладка не	
определена.	
3 Выполнение работ по ПМ.01	16
3.1 Проектирование программного обеспечения.....	16
3.2 Разработка программных модулей	19
3.3 Отладка и тестирование программных модулей.....	24
3.4 Оптимизация и рефакторинг программного кода.....	26
Заключение	29
Список использованных источников	30

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем техническом отчете применяются следующие сокращения и обозначения:

БД – база данных

ВУ – водительское удостоверение

ПК – персональный компьютер

ПМ – программный модуль

ПО – программное обеспечение

СУБД – система управления базами данных

ТЗ – техническое задание

ТС – транспортное средство

DDL – язык описания данных

ERD – диаграмма «сущность-связь»

ER-модель – модель «сущность-связь»

FK – внешний ключ

PK – первичный ключ

SQL – язык структурированных запросов

UI – интерфейс пользователя

VIN – идентификационный номер транспортного средства

VS2022 – Visual Studio 2022

XAML – расширяемый язык разметки для приложений

ВВЕДЕНИЕ

Производственная практика проводится на базе АРОО «Федерация тхэквондо ГТФ». Одной из задач предприятия является подготовка спортсменов к выступлению на соревнованиях. В рамках практики решается задача автоматизации учёта посещаемости тренировок в федерации и формирования групп спортсменов с целью замены журналов.

Производственная практика является неотъемлемой частью учебного процесса образовательной программы среднего профессионального образования по специальности 09.02.07 Информационные системы и программирование.

Целью прохождения производственной практики является получение практического опыта выполнения работ по ПМ.02 «Осуществление интеграции программных модулей» и ПМ.04 «Сопровождение и обслуживание программного обеспечения компьютерных систем» и развитие общих и профессиональных компетенций.

Для достижения поставленной цели требуется решить следующие задачи:

- проанализировать функциональные и эксплуатационные требования к ПО;
- сформировать требования к программным модулям по предложенной документации;
- реализовать и интегрировать модули в ПО;
- использовать системы контроля версий;
- использовать инструментальные средства на этапе отладки программного продукта;
- разработать тестовые наборы и сценарии;
- провести тестирование программного модуля по определенному сценарию;

- провести инспектирование разработанных программных модулей на предмет соответствия стандартам кодирования;
- выполнить сбор и настройку конфигурации ПО компьютерных систем;
- провести инсталляцию ПО компьютерных систем;
- выполнить настройку отдельных компонентов ПО компьютерных систем;
- сформировать эксплуатационные характеристики ПО компьютерных систем;
- выполнить модификацию отдельных компонентов ПО в соответствии с потребностями заказчика.

Для практикантов предоставляется рабочее место с ПК и всем необходимым для работы аппаратным и ПО:

- процессор: 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz;
- системная плата: Katana GF76 11SC;
- видеокарта: Nvidia GeForce GTX 1650;
- оперативная память – 8ГБ;
- ОС: Windows 11 Pro;
- Visual Studio 2022;
- Microsoft SQL Server Management Studio 20;
- draw.io. Diagrams.

1 Охрана труда и техника безопасности при работе на ПК

1.1 Общие требования безопасности

К самостоятельной работе на ПК допускаются лица, прошедшие медосмотр, инструктаж по охране труда, обучение безопасным методам работы, проверку знаний по электробезопасности (1 группа) и не имеющие медицинских противопоказаний.

Работник должен знать о возможных вредных факторах:

- перегрев оборудования, некомфортная температура воздуха;
- повышенное электрическое поле, яркий свет;
- зрительное и статическое напряжение;
- монотонность труда.

Обязанности работника:

- соблюдать трудовую дисциплину, режим работы и отдыха;
- выполнять только порученные задачи;
- содержать рабочее место в порядке;
- немедленно сообщать о неисправностях;
- соблюдать правила пожарной и электробезопасности.

Нарушение требований влечёт ответственность по закону РФ.

1.2 Требования безопасности перед началом работы

Перед началом работы работник должен проветрить помещение, правильно расположить оборудование, убрать лишние предметы, провести визуальный осмотр проводов и розеток на наличие повреждений, а также очистить монитор и клавиатуру от пыли при необходимости.

1.3 Требования безопасности во время работы

Во время работы за ПК сотрудник обязан:

- поддерживать порядок на рабочем месте, не заслонять вентиляционные отверстия ПК;
- корректно завершать задачи при перерывах, настраивать комфортные параметры экрана;
- соблюдать регламентированные перерывы и выполнять физические упражнения.

Во время работы запрещается:

- касаться задней панели системного блока или переключать кабели при включённом питании;
- закрывать оборудование посторонними предметами, допускать скопление бумаг или попадание влаги;
- отключать питание во время работы;
- самостоятельно ремонтировать технику;
- работать ближе 50 см от монитора.

1.4 Требования охраны труда в аварийных ситуациях

В случаях обрыва проводов питания, неисправности заземления и других повреждений, появления гари, следует немедленно отключить питание и сообщить об аварийной ситуации руководителю.

При возникновении пожара, задымлении требуется:

- открыть запасные выходы из здания, обесточить электропитание;
- закрыть окна и прикрыть двери;
- немедленно сообщить по телефону «101» или «112» в пожарную охрану, оповестить работающих, поставить в известность руководителя подразделения, сообщить о возгорании на пост охраны;

- приступить к тушению пожара первичными средствами пожаротушения, если это не сопряжено с риском для жизни;
- организовать встречу пожарной команды;
- покинуть здание и находиться в зоне эвакуации.
- При несчастном случае требуется:
- немедленно организовать первую помощь пострадавшему и при необходимости доставить его в медицинскую организацию;
- принять меры по предотвращению развития чрезвычайной ситуации и воздействия травмирующих факторов на других лиц;
- сохранить до начала расследования несчастного случая обстановку, какой она была на момент происшествия, если это не угрожает жизни и здоровью других лиц и не ведёт к катастрофе, аварии или возникновению иных чрезвычайных обстоятельств, а в случае невозможности ее сохранения, зафиксировать сложившуюся обстановку (составить схемы, провести другие мероприятия).

1.5 Требования охраны труда по окончании работы

После окончания работы работник обязан:

- произвести закрытие всех выполняемых на ПК задач;
- отключить питание в последовательности, установленной инструкциями по эксплуатации на оборудование, с учетом характера выполняемых работ;
- привести в порядок рабочее место.

2 Выполнение работ по ПМ.02

2.1 Анализ и разработка требований

Требуется разработать ИС для учета посещаемости тренировок в федерации и формирования групп спортсменов. Система должна заменить бумажные журналы учета посещаемости тренировок спортсменами, обеспечить снижение временных потерь в ходе проведения занятий в федерации.

Основные функциональные требования:

- авторизация пользователей (тренер);
- создание записей в БД о новых залах, группах, тренерах, спортсменах, тренировках;
- просмотр и редактирование данных о тренерах, залах, групп залах, группах, тренерах, спортсменах, тренировках;
- сортировка списка данных о тренировках по группам;
- фиксация посещаемости тренировок спортсменами.

2.2 Выбор состава программных и технических средств

Согласно требованиям необходимо создать ИС, позволяющую вести учет посещаемости тренировок.

Необходимо создать оконное приложение, работающее на ОС Windows. Данное решение было принято в связи с тем, что каждый кабинет тренера оснащается ноутбуком.

В качестве основных инструментов разработки выбрана платформа .NET 8 с использованием WPF и язык программирования C#, являющийся приоритетным языком для .NET.

Для разработки приложения будет использоваться IDE Visual Studio 2022, обладающая следующими преимуществами:

- встроенные средства отладки и тестирования кода;
- автогенерация кода;
- статический анализ кода;
- встроенные средства рефакторинга кода;
- встроенный мониторинг системных ресурсов;
- интеграция с системами управления версиями.

Для хранения данных выбрана СУБД Microsoft SQL Server, обеспечивающая надежное хранение информации, поддержку транзакций и целостность данных.

К целевым устройствам предъявляются следующие минимальные системные требования:

- ОС: Windows 10 и выше;
- оперативная память: 2 ГБ;
- 500 МБ свободного места на устройстве.

2.3 Проектирование программного обеспечения

Проектирования ПО определяет архитектуру системы, выбор технологий, методы и средства реализации функциональных требований и пользовательских сценариев.

В ходе проектирования ПО принято решение разработать физическую модель БД и созданы макеты пользовательского интерфейса для приложения.

БД должна быть спроектирована с учетом масштабируемости и предметной области.

Физическая модель БД, спроектированная с учетом предметной области, представлена на рисунке 1.

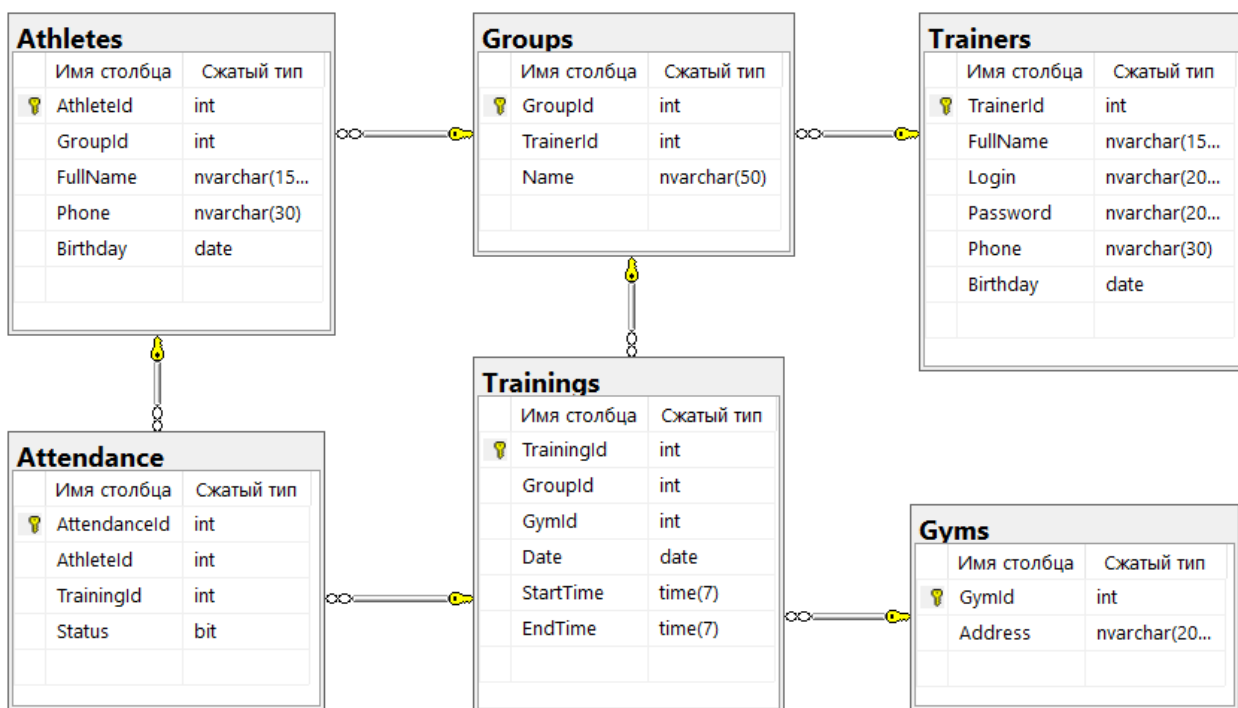


Рисунок 1 – «SSMS 20». Физическая модель БД

В ходе проектирования пользовательского интерфейса с помощью сервиса draw.io разработан каркас высокой точности wireframe для приложения.

На рисунке 2 отображена схема взаимодействия с экраном залов, на которой располагаются:

- список всех залов;
- кнопка перехода на экран «Добавление зала»;
- кнопка перехода на экран «Личные данные»;
- метка с ФИО тренера;
- кнопка «назад».

В процессе проектирования поставлена задача смоделировать интерфейс с интуитивно понятной последовательной навигацией. Это необходимо для удовлетворения пользователей в удобном и эффективном взаимодействии с приложением.

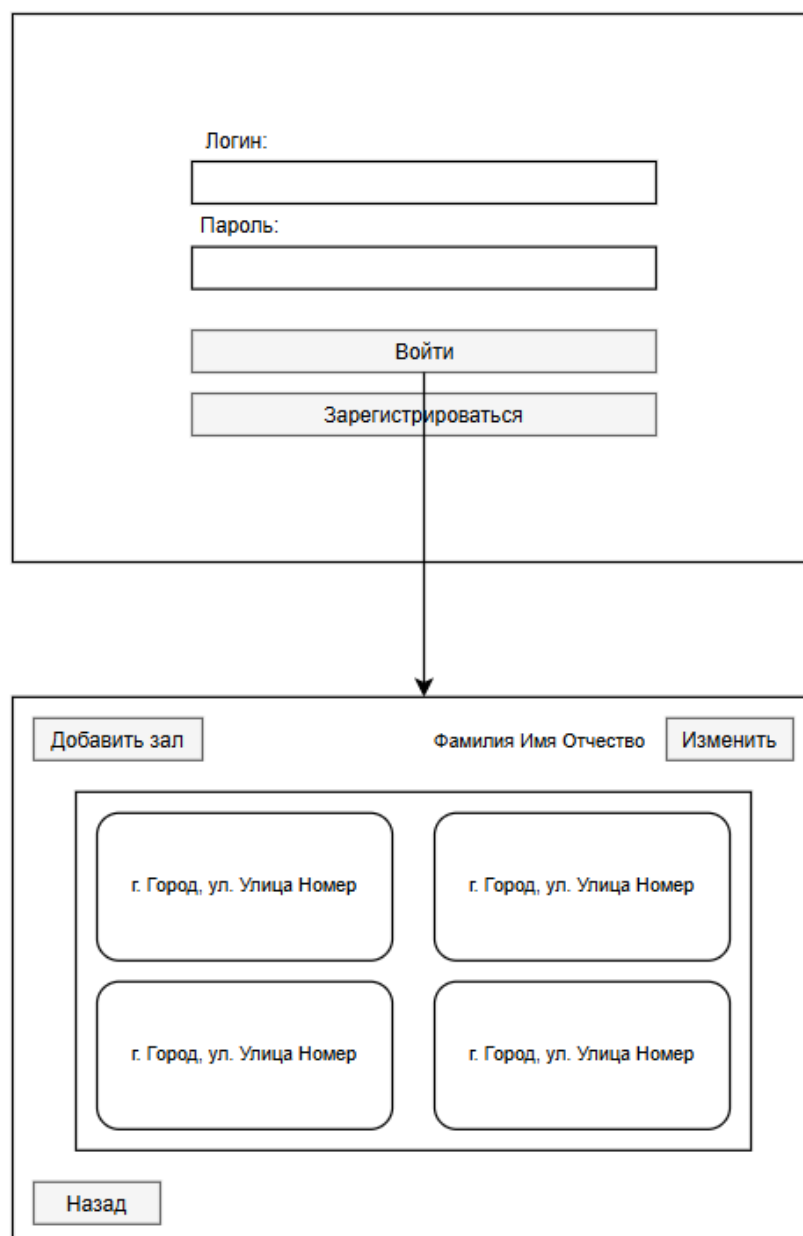


Рисунок 2 – «draw.io». Wireframe экрана «Залы»

2.4 Разработка и интеграция программных модулей

В ходе прохождения производственной практики разработано оконное приложения для тренеров на языке программирования C# в IDE VS.

Для разработки использовались следующие технологии, библиотеки и пакеты:

- WPF – технология построения графического интерфейса, позволяющая создавать десктопные приложения с использованием XAML;
- Microsoft.EntityFrameworkCore.SqlServer – провайдер Entity Framework Core для SQL Server, обеспечивающий подключение к реляционной БД и выполнение запросов;
- Microsoft.EntityFrameworkCore.Tools – инструменты командной строки для создания миграций, обновления схемы БД и управления контекстом данных;
- BCrypt.Net-Net – библиотека для хэширования паролей, применяемая при регистрации тренеров и проверке учетных данных при авторизации.

2.5 Описание работы с системой контроля версий и проверка соответствия стандартам кодирования

В процессе разработки ПО для контроля версий использовалась распределенная система контроля версий Git. В качестве модели ветвления проекта выбрана модель «Git Flow», в которой используются функциональные ветки (feature-ветки) для разработки и основная ветка для хранения стабильной версии проекта.

Для работы с Git использовалось приложение «GitHub Desktop» для Windows и встроенный модуль системы контроль версий VS. Для управления удаленным репозиторием Git использовался веб-сервис GitHub. Такой подход обеспечил параллельную разработку новых функций без риска нарушения стабильности основного кода.

Фрагмент графа коммитов в репозитории представлен на рисунке 3.



Публикация + установка	Kolosov Vadim	28.3.26 23:57:14	7de8601f
Стиль, компоновка	Kolosov Vadim	26.3.26 17:44:32	8149051b
Шифрование паролей, рефакторинг	Kolosov Vadim	25.3.26 17:46:49	619d3b47
Реализована сортировка тренировок по группам	Kolosov Vadim	24.3.26 19:57:17	a97d1ee3
Реализован ряд функций для работы с группами и спортсменами	Kolosov Vadim	23.3.26 22:52:52	1cdefa68
Реализация функции фиксации посещаемости	Kolosov Vadim	22.3.26 0:27:13	57c67e51
Реализована функция изменения и удаления тренировок	Kolosov Vadim	20.3.26 17:17:28	2d814e43
Реализована функция добавления тренировки	Kolosov Vadim	19.3.26 19:10:54	cb3adabe
Реализована функция добавления зала	Kolosov Vadim	18.3.26 17:23:27	295e7b2c
Функция удаления информации о зале из БД	Kolosov Vadim	18.3.26 17:08:46	7d835a9a
Функция обновления данных о зале	Kolosov Vadim	18.3.26 16:54:04	f75df847
Обновление данных о тренере	Kolosov Vadim	18.3.26 16:40:14	ce15d132
Реализована страница посещения	Kolosov Vadim	17.3.26 20:26:22	0f995a4e
Тренировки	Kolosov Vadim	17.3.26 19:09:01	d5e36946
Разработана функциональность авторизации	Kolosov Vadim	11.3.26 18:04:53	73c0dff2
Реализация страницы "Залы"	Kolosov Vadim	3.3.26 20:00:26	65e0a0c2
Разработана функциональность авторизации	Kolosov Vadim	3.3.26 19:04:53	d7d17d0e
Добавьте файлы проекта.	Kolosov Vadim	2.3.26 17:19:28	1dd1d6d2
Добавить .gitattributes и .gitignore.	Kolosov Vadim	2.3.26 17:19:26	98286b88

Рисунок 3 – Фрагмент графа коммитов в репозитории

В ходе производственной практики код и его структура разработаны в соответствии с соглашением о написании кода на языке программирования C#. При разработке соблюдены правила именования пространства имен, классов, методов, свойств и форматирования кода.

2.6 Отладка и тестирование программных модулей

Для отладки кода использовались следующие инструменты IDE VS:

- окно просмотра переменных;
- диагностические инструменты;
- окно потоков;
- анализ стека вызовов;
- точки останова.

На рисунке 4 показан общий вид IDE с установленной точкой останова в методе создания новой записи о зале в БД.



Рисунок 4 – «VS». Вид IDE с установленной точкой останова

Для проверки предсказуемости поведения приложения необходимо провести тестирование методом «черного ящика». В таблице 1 представлен набор тестов для страницы работы с группами.

Таблица 1 – Набор тестов страницы работы с группами

Действие	Ожидаемый результат	Полученный результат
В поле ввода написать «Младшая», затем нажать на кнопку «Создать группу»	Вывод ошибки «Группа с таким названием уже существует»	Совпадает с ожидаемым
В поле ввода написать «Вечерняя», затем нажать на кнопку «Создать группу»	Вывод сообщения «Группа создана.», в выпадающем списке выбрана созданная группа.	Совпадает с ожидаемым
В списке групп выбрать «Вечерняя», затем нажать «Удалить выбранную группу», далее нажать «Да»	Группа «Вечерняя» удалена и отсутствует в выпадающем списке групп	Совпадает с ожидаемым
В списке групп выбрать «Старшая», затем нажать «Удалить выбранную группу»	Вывод ошибки «Для этой группы уже создана тренировка, ее нельзя удалить.»	Совпадает с ожидаемым
Нажать на кнопку «Назад»	Переход на страницу «Личные данные»	Совпадает с ожидаемым

В результате тестирования не выявлено отклонений полученных результатов от ожидаемых.

3 Выполнение работ по ПМ.01

3.1 Проектирование программного обеспечения

ООО «Кактус» активно использует современные технологии цифрового мониторинга для оперативного выявления дорожных инцидентов, что является важной частью аналитической работы компании. В условиях интенсивного дорожного движения и большого потока информации требуется оперативная система сбора данных о происшествиях, позволяющая получать актуальные сведения в режиме реального времени.

Осознавая важность оперативного реагирования, компания поставила задачу по созданию специализированного программного обеспечения для автоматизированного сбора информации о дорожных инцидентах. Разрабатываемая система должна обеспечивать не только мгновенное оповещение о происшествиях, но и предоставлять полные структурированные данные, включая время, место и обстоятельства каждого инцидента. Это позволит проводить глубокий анализ причин и последствий происшествий, выявлять проблемные участки дорожной сети и разрабатывать эффективные меры по повышению безопасности дорожного движения. Внедрение такой системы значительно повысит качество аналитической работы и эффективность принимаемых управленческих решений [2].

В связи с этим ключевой задачей становится разработка специализированного приложения для оперативного мониторинга дорожных инцидентов с удобным оконным интерфейсом.

Система должна решать следующие задачи:

- оперативная фиксация данных о дорожных происшествиях;
- просмотр зарегистрированных инцидентов;
- добавление нового водителя в систему;
- добавление нового ТС в систему.

Функциональные возможности системы для оператора отображены на диаграмме прецедентов (рисунок 4). Приложение позволит не только оперативно регистрировать происшествия, но и проводить их последующий анализ для выявления опасных участков дорожной сети.

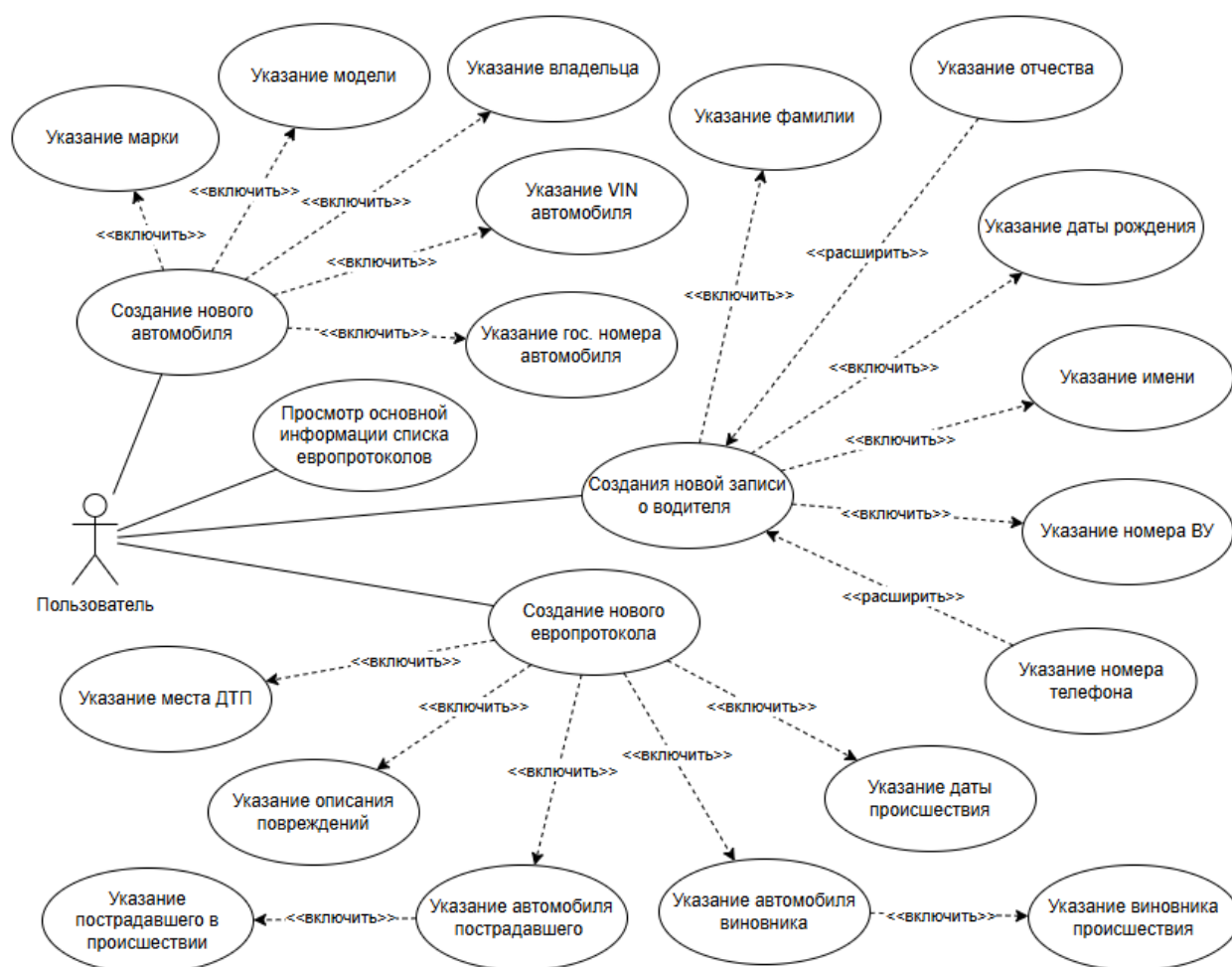


Рисунок 1 – Диаграмма прецедентов

При отображении перечня ТС виновника в выпадающем списке необходимо обеспечить фильтрацию автомобилей после выбора ответственного водителя. Данный механизм позволяет выводить в выпадающем списке исключительно те ТС, которые закреплены за указанным водителем. Такой подход к фильтрации данных значительно улучшает удобство взаимодействия с интерфейсом, особенно при работе с обширными

списками записей. Визуальное представление алгоритма фильтрации приведено на рисунке 5.

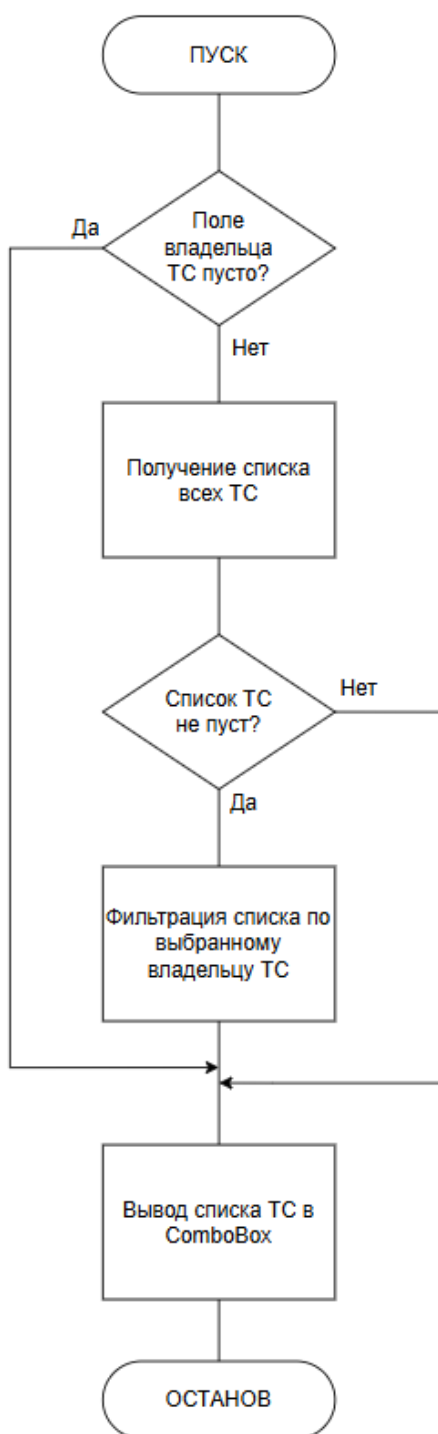


Рисунок 5 – Схема алгоритма фильтрации данных

3.2 Разработка программных модулей

Для решения поставленной задачи необходимо использовать язык программирования C#, подходящий для разработки разнообразных приложений. Для разработки кроссплатформенного приложения используется фреймворк Avalonia UI совместно с библиотекой Community Toolkit. При разработке UI приложения, используется архитектурный паттерн MVVM.

Для обеспечения хранения данных о европротоколах требуется разработать класс с полями, представляющими данные задачи. Код класса EuroprotocolDto представлен листингом 8.

Листинг 8 – Код класса EuroprotocolDto

```
public class EuroprotocolDto
{
    public int Id { get; set; } //Id европротокола

    public int CulpritCarId { get; set; } // Id ТС виновника

    public int VictimCarId { get; set; } // Id ТС пострадавшего

    public DateTime Date { get; set; } // Дата и время
    происшествия

    public string Place { get; set; } = null!; // Место
    происшествия

    public string Description { get; set; } = null!; // Описание
    повреждений
}
```

Так же для загрузки всех автомобилей у выбранного пострадавшего в ДТП необходимо создать метод LoadVictimCars (листинг 9). Данный метод осуществляет выборку транспортных средств из базы данных на основе идентификатора пострадавшего водителя, обеспечивая корректное отображение только принадлежащих ему автомобилей.

Листинг 9 – Метод LoadVictimCars

```
private async void LoadVictimCars()
{
    // Проверка выбранного виновника на null значение
    if (SelectedVictim == null) return;

    // Получение всех автомобилей
    var cars = await _dataService.CarService.GetAllAsync();

    // Сортировка авто
    if (cars != null)
    {
        VictimCars = cars.Where(c => c.DriverId ==
SelectedVictim.Id).ToList();
    }
}
```

Для управления информацией о европротоколах необходимо разработать класс `EuroprotocolService`, который предоставляет функциональность для работы с данными, хранящимся в БД, а именно: получение, добавление, удаление и редактирование записей о задачах [5]. Фрагмент кода, демонстрирующий получение данных с использованием класса `EuroprotocolService`, представлен в листинге 10.

Листинг 10 – Фрагмент кода класса EuroprotocolService

```
public class EuroprotocolService : IService<EuroprotocolDto>
{
    private readonly HttpClient _client;
    private readonly string _url =
"http://localhost:5180/api/Europrotocols";

    public EuroprotocolService(HttpClient client)
    {
        _client = client;
        _client.BaseAddress = new Uri(_url);
    }

    public EuroprotocolService()
    {
        _client = new HttpClient();
        _client.BaseAddress = new Uri(_url);
    }
}
```

```

// Метод получения всех европротоколов
public async Task<List<EuroprotocolDto?>> GetAllAsync()
    => await
_client.GetFromJsonAsync<List<EuroprotocolDto?>>("");

// Метод получения европротокола по его Id
public async Task<EuroprotocolDto?> GetAsync(int id)
    => await
_client.GetFromJsonAsync<EuroprotocolDto?>($"{{id}}");
}

```

Необходимо разработать XAML-разметку главной страницы, точно соответствующую макету на рисунке 6. Интерфейс должен включать основные элементы: панель навигации, список инцидентов.

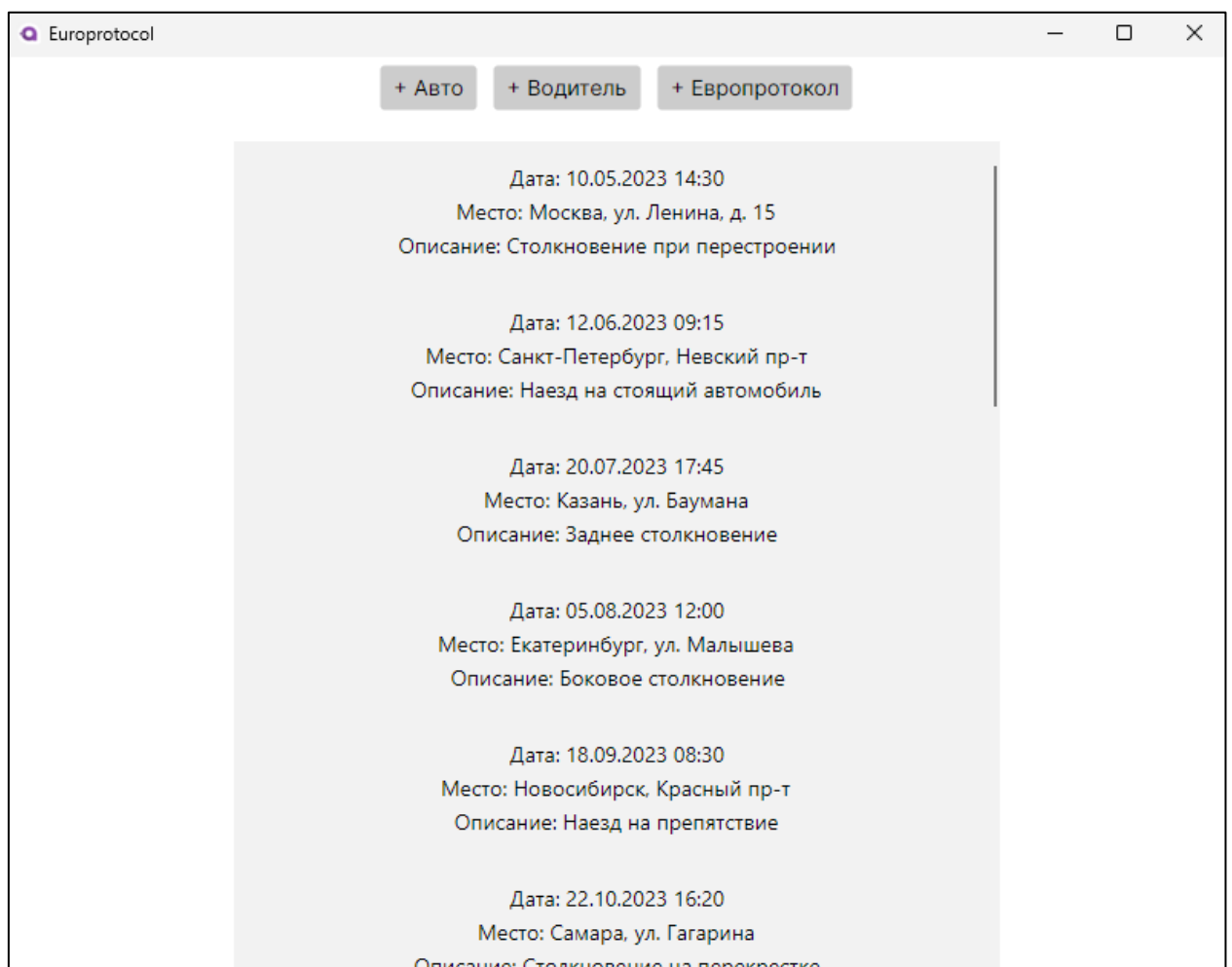


Рисунок 6 – Europrotocol. Вид главной страницы

Используя XAML, требуется спроектировать разметку формы для добавления новой записи о водителе в соответствии с рисунком 7.

При проектировании разметки формы добавления нового водителя в систему необходимо предусмотреть следующие ключевые элементы интерфейса: поля ввода для фамилии, имени, отчества и номера ВУ.

The screenshot shows a window titled "Europrotocol" with three buttons at the top: "+ Авто", "+ Водитель", and "+ Европротокол". The main form is titled "Добавление водителя" and contains the following fields:

- Фамилия
- Имя
- Отчество
- 4 | июнь | 2025
- Водительское удостоверение
- Телефон
- Сохранить

Below the form, a list of records is displayed:

- Дата: 10.05.2023 14:30
Место: Москва, ул. Ленина, д. 15
Описание: Столкновение при перестроении
- Дата: 12.06.2023 09:15
Место: Санкт-Петербург, Невский пр-т
Описание: Наезд на стоящий автомобиль
- Дата: 20.07.2023 17:45

Рисунок 7 – Europrotocol. Вид формы добавления записи о новом водителе

Для реализации функции перехода на форму заполнения европротокола необходимо создать метод `OpenAddEuroprotocol` в `MainViewModel`. Метод должен осуществлять отображение формы добавления европротокола на экране. Код метода `OpenAddEuroprotocol` представлен листингом 11.

Листинг 11 - Код метода OpenAddEuroprotocol

```
//Метод для отображения формы заполнения европротокола
void OpenAddEuroprotocol() => CurrentView =
    new AddEuroprotocolView
    { DataContext = new AddEuroprotocolViewModel() };
```

После создания метода `OpenAddEuroprotocol`, необходимо реализовать метод `SaveAsync` для сохранения европротокола в БД. Код метода `SaveAsync` представлен листингом 12.

Листинг 12 – Код метода SaveAsync

```
// Метод сохранения европротокола в бд
[RelayCommand]
private async Task SaveAsync()
{
    // Проверка на пустые поля
    if (SelectedCulpritCar.Id == null ||
        SelectedVictimCar.Id == null ||
        Date == null ||
        string.IsNullOrWhiteSpace(Place) ||
        string.IsNullOrWhiteSpace(Description))
        return;
    // создание нового европротокола
    var europrotocol = new EuroprotocolDto
    {
        CulpritCarId = SelectedCulpritCar.Id,
        VictimCarId = SelectedVictimCar.Id,
        Date = Date.DateTime,
        Place = Place,
        Description = Description
    };

    // сохранение европротокола в БД
    await
    _dataService.EuroprotocolService.AddAsync(europrotocol);
    _mainViewModel.Europrotocols =
    new ObservableCollection<EuroprotocolDto>
        (await _dataService.EuroprotocolService.GetAllAsync());
    Cancel();
}
```

3.3 Отладка и тестирование программных модулей

Для отладки приложения применяется комплекс инструментов VS2022, обеспечивающий диагностику и локализацию ошибок, а также внесение изменений для их устранения.

Стандартные средства отладчика VS2022:

- установка точки останова нажатием F9 (при расположении курсора на необходимой строке);
- включение и отключение точек останова комбинацией CTRL+F9;
- запуск отладки с остановками на точках останова нажатием F5;
- пошаговый переход без захода и с заходом в методы с помощью клавиш F10 и F11.

Для проверки работы приложения необходимо провести тестирование.

Для формирования теста необходимо указать выполняемое действие, ожидаемый результат и полученный результат. В таблице 2 приведен набор тестов разработанного приложения [3].

Таблица 2 – Набор тестов приложения

Действие	Ожидаемый результат	Полученный результат
Нажать на кнопку «+ Европротокол»	Переход на форму заполнения европротокола	Соответствует ожидаемому
Нажать на кнопку «+ Европротокол», нажать на кнопку «Отменить»	Закрытие формы заполнения европротокола	Соответствует ожидаемому
Нажать на кнопку «+ Европротокол», заполнить все поля, нажать на кнопку «Сохранить»	Закрытие формы заполнения европротокола, отображение только что созданной записи в списке всех европротоколов	Соответствует ожидаемому
Нажать на кнопку «+ Авто», заполнить все поля, нажать на кнопку «Сохранить», нажать на кнопку «+ Европротокол», заполнить все поля	Закрытие формы добавления авто после нажатия на кнопку «Сохранить», наличие созданного ТС в выпадающих списках ТС	Соответствует ожидаемому

Также для тестирования программы был разработан автоматизированный Unit-тест (представлен листингом 14) для проверки корректности работы метода `SaveAsync` в модели данных `AddDriverViewModel`. Код метода `SaveAsync` представлен в листинге 13.

Листинг 13 – Код метода `SaveAsync`

```
[RelayCommand]
public async Task SaveAsync()
{
    // Проверка полей на null значения
    if (string.IsNullOrEmpty(Name) ||
        string.IsNullOrEmpty(Surname) ||
        Birthday == null ||
        String.IsNullOrEmpty(DriverLicense))
        return;

    // Создание объекта класса водителя
    var driver = new DriverDto
    {
        Name = Name,
        Surname = Surname,
        Patronymic = Patronymic,
        Birthday = Birthday.Date,
        DriverLicense = DriverLicense,
        Phone = Phone
    };

    // Сохранение водителя
    await _dataService.DriverService.AddAsync(driver);
    _mainViewModel.CurrentView = null;
}
```

Листинг 14 – Код автоматизированного Unit-теста

```
[Fact]
public async Task SaveAsync_WithValidData_ShouldAddDriver()
{
    // Arrange
    var mockDriverService = new Mock<DriverService>();
    // Создаем реальный DataService с подменным DriverService
    var dataService = new DataService
    {
        DriverService = mockDriverService.Object
    };
};
```

```

var mockMainViewModel = new Mock<MainViewModel>();

// Создание нового объекта с тестовыми полями
var viewModel = new AddDriverViewModel(dataService,
mockMainViewModel.Object)
{
    Name = "Иван",
    Surname = "Иванов",
    Birthday = new DateTime(1990, 1, 1),
    DriverLicense = "1234567890"
};

mockDriverService.Setup(s =>
s.AddAsync(It.IsAny<DriverDto>())) .Returns(Task.CompletedTask);

// Act
await viewModel.SaveAsync();

// Assert
mockDriverService.Verify(s => s.AddAsync(It.Is<DriverDto>(d
=>
    d.Name == "Иван" &&
    d.Surname == "Иванов" &&
    d.Birthday == new DateTime(1990, 1, 1).Date &&
    d.DriverLicense == "1234567890"
)), Times.Once);
}

```

В ходе проведения автоматизированного тестирования результаты теста показали, что метод `SaveAsync` работает корректно. Изменений вносить не требуется.

3.4 Оптимизация и рефакторинг программного кода

В ходе оптимизации были выявлены уязвимости, которые влияют на производительность.

Для отображения данных в выпадающих списках об автомобилях конкретного водителя используются методы `LoadCulpritCars` и `LoadVictimCars`. В обоих методах есть дублирующийся код, что излишне и может оказывать нагрузку на систему [4].

При загрузке всех автомобилей в выпадающий список используется код, представленный листингом 15.

Листинг 15 – Фрагмент кода обработки загрузки ТС с дублированием кода

```
private async void LoadCulpritCars()
{
    // Проверка водителя на null значение
    if (SelectedCulprit == null) return;

    // Получение всех автомобилей
    var cars = await _dataService.CarService.GetAllAsync();

    // Сортировка ТС
    if (cars != null)
    {
        CulpritCars = cars.Where(c => c.DriverId ==
SelectedCulprit.Id).ToList();
    }
}
private async void LoadVictimCars()
{
    // Проверка водителя на null значение
    if (SelectedVictim == null) return;

    // Получение всех автомобилей
    var cars = await _dataService.CarService.GetAllAsync();

    // Сортировка ТС
    if (cars != null)
    {
        VictimCars = cars.Where(c => c.DriverId ==
SelectedVictim.Id).ToList();
    }
}
```

Для оптимизации данного кода следует разработать и использовать отдельный метод LoadDriverCars, код представлен листингом 16.

Листинг 16 – Код метода загрузки автомобилей LoadDriverCars

```
// Универсальный метод получения нужных авто с сортировкой
private async Task<List<CarDto>> LoadDriverCars(int? driverId) {
    if (driverId == null) return new List<CarDto>();
    // Получение и сортировка всех авто
```

```

        var cars = await _dataService.CarService.GetAllAsync();
        return cars?.Where(c => c.DriverId == driverId).ToList() ??
new List<CarDto>();
    }

```

Также в ходе рефакторинга было обнаружено несоответствие возвращаемых типов данных и названий у метода Save в AddCarViewModel. При использованиях асинхронного кода в разрабатываемых методах сохранения были использованы синхронные вызовы, сигнатура данных методов была изменена для асинхронных вызовов. Код всех изменений после оптимизации и рефакторинга представлен в листинге 17

Листинг 17 – Код метода сохранения SaveAsync с исправлениями

```

[RelayCommand]
private async Task SaveAsync()
{
    // Проверка всех полей на null значения
    if (string.IsNullOrEmpty(Brand) ||
string.IsNullOrEmpty(Model) ||
        SelectedDriver.Id == null ||
string.IsNullOrEmpty(StateNumber) ||
        string.IsNullOrEmpty(Vin))
        return;
    // Создание нового объекта класса CarDto
    var car = new CarDto
    {
        Brand = Brand,
        Model = Model,
        DriverId = SelectedDriver.Id,
        StateNumber = StateNumber,
        Vin = Vin
    };
    // Сохранение авто в бд
    await _dataService.CarService.AddAsync(car);
    _mainViewModel.CurrentView = null;
}

```

ЗАКЛЮЧЕНИЕ

Прохождение производственной практики в ООО «Кактус» позволило достичь ключевых целей профессионального обучения и получить практический опыт в области разработки ПО. В ходе работы были успешно реализованы все поставленные задачи, что способствовало развитию профессиональных компетенций в соответствии с современными требованиями информационных технологий. В частности, были достигнуты следующие цели:

- получен практический опыт по выполнению работ по ПМ.11 «Разработка, администрирование и защита баз данных» и развитию общих и профессиональных компетенций;
- получен практический опыт по выполнению работ по ПМ.01 «Разработка модулей программного обеспечения для компьютерных систем» и развиты общие и профессиональные компетенций.

Для достижения целей практики выполнены следующие задачи:

- осуществлён сбор, обработка и анализ информации при проектировании БД;
- спроектирована БД;
- разработаны объекты БД;
- реализована БД в конкретной СУБД;
- решены вопросы администрирования БД;
- реализованы методы и технологии защиты информации в БД;
- сформированы алгоритмы разработки ПМ в соответствии с ТЗ;
- разработаны ПМ в соответствии с ТЗ;
- выполнена отладка ПМ с использованием специализированных программных средств;
- выполнено тестирование ПМ;
- осуществлен рефакторинг и оптимизация программного кода;
- разработаны модули ПО для мобильных платформ.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Голицына, О. Л. Базы данных : учебное пособие / О. Л. Голицына, Н. В. Максимов, И. И. Попов. – 4-е изд., перераб. и доп. – Москва : ФОРУМ : ИНФРА-М, 2023. – 400 с. – (Высшее образование: Бакалавриат). – URL: <https://znanium.om/catalog/product/1937956> (дата обращения: 10.05.2025). – Режим доступа: по подписке. – Текст : электронный.

2. Голицына, О. Л. Программное обеспечение : учебное пособие / О. Л. Голицына, Т. Л. Партыка, И. И. Попов. – 4-е изд., перераб. и доп. – Москва : ФОРУМ : ИНФРА-М, 2021. – 447 с. : ил. – (Профессиональное образование). – Текст : электронный. – URL: <https://znanium.ru/catalog/product/1189345> (дата обращения: 02.05.2025). – Режим доступа: по подписке.

3. Плаксин, М. А. Тестирование и отладка программ для профессионалов будущих и настоящих. – 4-е изд., электрон. – Москва : Лаборатория знаний, 2020. – 170 с. – URL: <https://ibooks.ru/bookshelf/353395/reading> (дата обращения: 24.05.2025). – Режим доступа: для зарегистрир. пользователей. – Текст: электронный.

4. Фленов, М. Е. Библия C#. – 6-е изд., перераб. и доп. / М. Е. Фленов. – Санкт-Петербург : БХВ-Петербург, 2024. – 512 с. – URL: <https://ibooks.ru/bookshelf/396461/reading> (дата обращения: 15.05.2025). – Режим доступа: для зарегистрир. пользователей. – Текст: электронный.

5. Хорев, П. Б. Объектно-ориентированное программирование с примерами на C#. – Москва : Форум, 2020. – 200 с. – URL: <https://ibooks.ru/bookshelf/361450/reading> (дата обращения: 11.05.2025). – Режим доступа: для зарегистрир. пользователей. – Текст: электронный.